TLF: Transactional Lock Fusion

Guy E. Blelloch
Carnegie Mellon University
Pittsburgh, PA, USA
guyb@cs.cmu.edu

Zachary Kent
Carnegie Mellon University
Pittsburgh, PA, USA
zkent@cs.cmu.edu

Yuanhao Wei
University of British Columbia
Vancouver, BC, Canada
yuanhaow@cs.ubc.ca

ABSTRACT

Software Transactional Memory (STM) systems have made many advances over the past decades, and data structures that use STM are approaching the efficiency of hand-designed concurrent data structures. Hand-designed structures, however, still maintain a key advantage over implementations with STMs: with careful design, they can ignore "unimportant" read-write conflicts. Some of the most efficient concurrent structures, for example, are based on optimistic locking (OL). Operations that use OL have a traversal phase with no locks, where read-write conflicts are ignored, and a commit phase where conflicts are protected with fine grained locks. An STM, on the other hand, does not know which conflicts are important and will serialize all reads and writes, potentially at a significant cost.

In this paper we describe an approach for taking OL data structures, pretty much as is, and incorporating them into a variety of STM systems. The approach allows fusing any number of OL operations in an atomic transaction, and, importantly, the OL operations will still ignore the unimportant conflicts. Moreover, it leverages the often significant effort the designer of an OL data structure has put into ensuring correctness. We refer to the approach as transactional lock fusion (TLF). We define what constitutes an OL data structure, define the TLF protocol, prove the TLF protocol maintains the correctness of the OL structure, and specify what is required of an STM system to support TLF.

We then implement the TLF protocol in two very different STMs: a single-version system based on two-phase locking, and a multiversion system based on optimistic locking. Both are state-of-the-art. We evaluate the effectiveness of the TLF protocols within these systems compared to pure STMs across several data structures and varying many parameters (size, update rate, distribution, transaction size, and number of processors). Our experiments show substantial speedup by using TLF over STM on most of the data structures.

CCS CONCEPTS

• **Computing methodologies** → **Concurrent algorithms**; **Concurrent computing methodologies**.

KEYWORDS

concurrent data structures, snapshots, locks

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
SPAA '25, July 28–August 1, 2025, Portland, OR, USA
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1258-6/2025/07.
https://doi.org/10.1145/3694906.3743341

1 INTRODUCTION

Early software transactional memory implementations suffered from high overheads and various limits in functionality [8]. Over the years, however, researchers have suggested many improvements. Such improvements include removing levels of indirection, supporting opacity [12], more efficient locks [34], better contention management [37], efficient multiversioning [27, 29, 31, 41], efficient time-stamping [12, 33], and efficient memory management [27, 39]. Today STMs are significantly more competitive.

However, hand designed concurrent data structures still have one important advantage over implementations with STMs—with careful design they can avoid unnecessary memory conflicts. In particular many, or even most, modern concurrent data structures have a read-only traversal phase which is performed where memory reads are unprotected by any locks or other synchronization, and then a commit phase performed under locks or with atomic instructions (e.g., compare and swap). When using locks this approach is often called optimistic locking (OL) and is widely used [1, 4, 5, 11, 18, 23, 26, 32]. An STM system, at least in its pure form, does not know when it is safe to avoid synchronization and hence needs to protect all read-write conflicts, inducing the conflicts the data structure designer carefully ignored. In some cases such synchronization can drastically decrease performance and scalability.

To alleviate such unnecessary synchronization, various researchers have suggested approaches to extend transactional memory systems beyond just reads and writes so that they can directly incorporate higher-level operations. An ideal implementation would allow batches of higher-level operations to be executed atomically as efficiently as if they were run separately, assuming low contention. These approaches, however, either require special-purpose implementations of the higher-level operations [17, 21, 30, 38], or have limited scope [6, 13, 19, 36, 42]. See the related work in Section 9 for a full discussion.

In this paper, we describe an approach for taking concurrent data structures based on optimistic locking (OL), pretty much as is, and incorporate them into an STM system. The approach effectively allows *fusing* of the data structure operations so any number of them will happen atomically. Like the original structures, however, it avoids any synchronization during the traversal phases. The approach can be used with a broad set of STM systems, and allows the user to combine standalone uses of OL operations, along with those in a transaction. We refer to the approach as *transactional lock fusion* (TLF).

At a high level the TLF protocol is rather straightforward. In the unprotected regions (traversal) of an OL operation, reads are unprotected and therefore do not require a read lock in pessimistic STMs or validation in optimistic STMs. This is where the savings come from. On the other hand, the composition of all the protected regions in a transaction serialize with respect to other transactions.

The correctness of the TLF protocol, however, is non-obvious—there are many subtle points. Indeed, the requirements of an OL implementation is not even precisely defined. We therefore carefully define what it means to be an OL-based implementation. The OL requirement is, roughly, that (1) all OL operations consist of protected (e.g., by locks) and unprotected regions, (2) the protected regions serialize, and (3) the execution of protected regions only depend on the protected regions that serialize earlier. All OL-based data structures we know of satisfy our OL requirements. We also demonstrate that proving a data-structure satisfies our OL conditions can be much simpler than proving the data structure is correct, and in particular it can mostly be done by examining the code.

We then define what it means to be a TLF protocol, and prove that TLF protocols correctly implement atomic transactions on sequences of OL operations. Furthermore we describe the properties needed by an STM system to integrate the system with TLF. Roughly, TLF requires that unprotected reads are safe, that writes are delayed until the transaction knows it will succeed, and that the composition of the protected regions in a transaction serialize.

Based on these conditions, we integrated TLF with two STM systems. The first is 2PLSF [34], a state-of-the-art starvation-free STM system based on two-phase locking, and the second is a multiversion STM using optimistic concurrency [24]. The two STM protocols are very different: optimistic vs. pessimistic, and single versioned vs. multiversioned. We needed to make some minor changes to 2PLSF to satisfy the requirements. In particular we added safe memory reclamation so unprotected reads are safe, and switched writes from write-through (immediate) to delayed. The optimistic STM already had these features.

To support libraries of OL operations we designed a common C++ interface for TLF, which we call Fuse, so the same code can be run any of our implementations. Modifying existing OL operations to use the interface mostly requires very simple syntactic changes. In cases where operations are unprotected by any lock, it requires adding a read lock so that all operations linearize within a lock. See Section 2 for an example on binary trees.

We perform extensive experiments that compare the original STM systems with the TLF-enhanced STM systems. We compare the performance of several data structures including a btree, a sorted list, a hash table, an AVL tree, a treap, a leaftree and a skiplist. The OL implementations are from the Verlib library [4], and we modified these for the STM implementations (e.g., removing locks, validation, retry loops, and various flags). We compare performance on a wide variety of workloads where we vary (1) the input size, (2) the distribution (via a zipfian parameter), (3) the update rate, (4) the transaction size, and (5) the number of threads. We both report aggregate geometric means across the parameters, showing how the implementations perform "on average" across workloads, and study how relative performance varies across each parameter.

Figure 1 shows the relative performance across our eight data structures for 2PLSF and MV each with and without TLF. See Section 10 for more details on the setup. As the figure shows, the advantage of TLF varies notably across the structures. In particular, the ART Tree, BTree and hash table are shallow so the traversal phase is very short. Hence the improvement due to TLF is small, although still 20% for 2PLSF. On the other hand, the traversal in the linked list is long and leads to a factor of more than 10x in

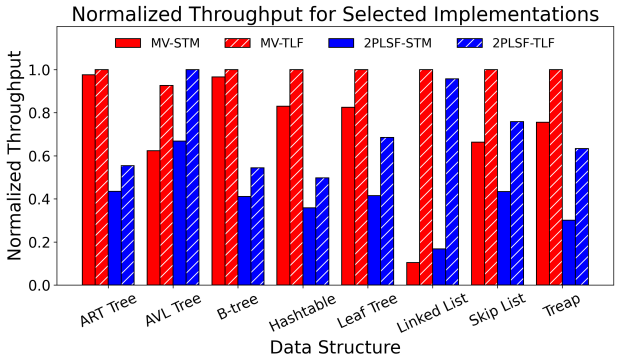


Figure 1: Comparison of throughput for eight data structures using 2PLSF and MV, each with and without TLF. Bars represent the geometric mean of throughput across three input sizes, two zipfian distributions, three update rates, and two transaction sizes, and normalized to the highest throughput (higher is better).

improvement. The other structures are binary trees and hence have $O(\lg n)$ depth. In these, the performance improvement ranges from about 30% to 200%. Both the 2PLSF STM and the MV STM benefit from TLF.

In the experimental section we also compare performance to the original optimistic locking code (without using an STM or TLF). The results show that we can get within 20-30% of the performance of the non-transactional code. We note these are state-of-the-art implementations.

The contributions of the paper include the following:

- We define the important properties of optimistic locking (OL) implementations.
- We introduce the transactional-lock-fusion (TLF) protocol and define its semantics independently of specific STM systems.
- We prove that the TLF protocol maintains correctness of OL implementations satisfying our requirements while serializing all operations within the transactions.
- We describe general approach of incorporating TLF into STM systems and present two examples of quite different systems that we have implemented.
- We experimentally evaluate the effectiveness across several data structures and workloads.

Our code for Fuse and the experiments can be found at:
https://github.com/cmuparlay/fuse

2 THE FUSE LIBRARY AND EXAMPLES

In this section we describe our C++-based Fuse library interface, which is implemented using TLF, and an example of how to integrate an optimistic locking data structure into the library. There are three levels in which one can use Fuse. At the top level (Figure 2) it acts as a standard software transactional memory (STM) interface, except that it includes a set of built-in data structure libraries. Code that is wrapped in an `atomic_region` will act atomically as long as

```
1  T* fuse::New(args...);  // use like 'new T(args...)'
2  void fuse::Retire(T* x); // use like 'delete x'

4  // f is a functor with no argument and returning T
5  T fuse::atomic_region(F f);
6  T fuse::atomic_read_only(F f);

8  struct fuse::tds_atomic<T>;
9  struct fuse::tds_btree_map<K,V>;
10 ...
```

Figure 2: The C++ Fuse top-level interface. The `fuse::atomic_region` and its read only counterpart take a function with no argument and run it in a transaction. Template declarations are dropped.

```
1  fuse::tds_btree_set set_1;
2  fuse::tds_btree_set set_2;
3  fuse::tds_atomic<int> counter;
4  ...
5  result = fuse::atomic_region([&] {
6    key x = ...
7    cnt = counter.load();
8    if (set_1.find(x) && !set_2.find(x)) {
9      set_2.insert(x);
10   else counter.store(cnt + 1);
11   return cnt;}}
```

Figure 3: Simple example of use of transaction with the optimistic lock-based operations.

all shared data is accessed through transactional data structures (e.g. `tds_atomic` and `tds_btree_map`). Furthermore, `fuse::New` and `fuse::Retire` are used instead of new and delete within atomic regions. The data structure libraries include `tds_atomic`, which supports loads and stores (i.e. traditional STM), as well as all the data structures described in Section 8. Figure 3 gives an example usage at this top level.

At the middle level (see Figure 4) the user can convert a structure based on (optimistic) locking into a library to be used with fuse. The conversion involves replacing the C++ standard template library versions of `atomic` and `shared_mutex` with the Fuse versions[1], replacing new and delete with `fuse::New` and `fuse::Retire` (as in the top level), and ensuring all operations linearize within a lock. In the simplest case `tds_atomic` can be implemented as shown in Figure 5. This simply uses `fuse::atomic` and protects the operations on it with locks so the operations linearize within a lock. We present a more involved example below. At the bottom level one can convert Fuse to use different underlying STM systems. This is outlined in Section 6. In our current implementation we support an optimistic multiversion system, and a pessimistic single version system. The system can be chosen using a compiler flag.

We now present a more involved example of using the middle level to convert an optimistic-locking based implementation of a set to be used with Fuse. The example is for a leaf-based binary tree (see Figure 6 for an illustration of a leaf tree and 7 for the code). The code is based on the `leaftree` implementation in the Verlib library [4]. It replaces the atomic types, locks, and allocation, with the Fuse versions. In addition to these syntactic changes, this particular code did require adding a read lock and validation in

[1]Presently, `fuse::atomic` is optimized for bool and pointer types.

```
1  struct fuse::atomic<T>; // use like std::atomic

3  struct fuse::shared_mutex; // use like std::shared_mutex
```

Figure 4: The middle-level interface for converting optimistic locking data structures to be used with Fuse. The `fuse::atomic` and `fuse::shared_mutex` are replacement for the C++ std versions.

```
1  template <typename T>
2  struct tds_atomic {
3    fuse::atomic<T> v;
4    fuse::shared_mutex lock;

6    T load() {
7      std::shared_lock lck(lock);
8      return v.load(); }

10   void store(T x) {
11     std::unique_lock lck(lock);
12     v.store(x);}};
```

Figure 5: Fuse code to support traditional STM loads and stores in a transaction. It simply protects each load and store with read or write locks, respectively.

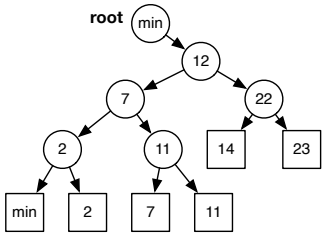


Figure 6: Example of a leaftree with keys 2, 7, 11, 14 and 23 stored at the leaves (squares). Deleting the key 14 would remove the leaf 14 and its parent 22, linking 12 to 23.

the find (Lines 30–32). This is to satisfy our requirement that all operations must linearize within a lock. Many OL structures already ensure that all operations linearize with a lock.

The implementation is for a tree where all the keys are kept at the leaves, and the internal nodes have copies of the keys used for guidance. In the algorithm, all operations (`find`, `insert`, and `remove`) first do a traversal to a leaf using `find_leaf` without taking any locks. This is a key aspect of OL algorithms. The `find_leaf` returns a tuple consisting of the grandparent, the parent and the leaf. The find, insert and remove then acquire a lock and validate that the local state has not changed—in particular, for find and insert, that the parent's child is still the same leaf and that the parent has not been deleted (Line 32 and Line 39), and for remove, that the grandparent's and parent's child are still the same, and the grandparent has not been deleted (Lines 55-57). If the validate fails, the operation is retried (i.e. the while loop is repeated). If the

```
struct tds_leaftree_set {
  struct node {
    bool is_leaf;
    Key key;
    fuse::atomic<bool> deleted;
    fuse::shared_mutex lock;
    fuse::atomic<node*> left;
    fuse::atomic<node*> right;
    node(Key key) : is_leaf(true), key(key) {}
    node(Key key, node* l, node* r)
      : is_leaf(false), key(key), left(l), right(r) {};};
  node* root;

  fuse::atomic<node*>* get_child(node* a, Key k) {
    return (k < a->key) ? &parent->left : &parent->right;}

  auto find_leaf(Key k) {
    node* parent = root;
    node* gnd_parent = nullptr;
    while (true) {
      node* child = get_child(parent, k)->load();
      if (child->is_leaf)
        return std::tuple(gnd_parent, parent, child);
      gnd_parent = parent;
      parent = child; }};

  bool find(Key k) {
    while (true) {
      auto [gnd_parent, parent, leaf] = find_leaf(k);
      std::shared_lock lck(parent->lock);
      auto leaf_ptr = get_child(parent, k);
      if (!parent->deleted && leaf_ptr->load() == leaf)
        return (leaf->key == k);}}

  bool insert(Key k) {
    while (true) {
      auto [gnd_parent, parent, leaf] = find_leaf(k);
      std::unique_lock lck(parent->lock);
      auto leaf_ptr = get_child(parent, k);
      if (!parent->deleted && leaf_ptr->load() == leaf) {
        if (leaf->key == k) return false;
        node* new_leaf = fuse::New<node>(k);
        if (leaf->key > k)
          leaf_ptr->store(fuse::New<node>(k, leaf, new_leaf));
        else leaf_ptr->store(fuse::New<node>(leaf->key,
                                             new_leaf, leaf));
        return true;}}}

  bool remove(Key k) {
    while (true) {
      auto [gnd_parent, parent, leaf] = find_leaf(k);
      std::unique_lock lck1(gnd_parent->lock);
      std::unique_lock lck2(parent->lock);
      auto g_ptr = get_child(gnd_parent, k);
      auto p_ptr = get_child(parent, k);
      if (!gnd_parent->deleted &&
          g_ptr->load() == parent &&
          p_ptr->load() == leaf) {
        if (leaf->key != k) return false;
        parent->deleted = true;
        ptr->store((k < parent->key) ? parent->right.load()
                                     : parent->left.load());
        fuse::Retire(leaf);
        fuse::Retire(parent);
        return true;}}}
};
```

Figure 7: C++ code for a concurrent ordered set based on a leaftree [1] augmented with Fuse. The algorithm uses optimistic locking. The `find`, `insert` and `remove` functions first traverse to a leaf using `find_leaf` without taking any locks, then take locks, and if the validate is successful, apply the update or return the value. If not successful, the operation is retried in the while loop. Code in `red` is what needs to be changed to use Fuse from the original. Code in `blue` might or might not appear in the original code but is needed in Fuse to ensure the `find` linearizes within a lock.

validate for the find succeeds, it returns whether the key at the leaf equals the search key in the leaf. If the insert or remove succeeds, it updates an internal node by either adding a new child (insert) or shortcutting out the old child (remove).

Importantly, the locks are only required for a short amount of time (constant number of instructions) and the locks that are taken are near the leaves of the tree (at most two levels up). Furthermore, a validation will only fail if a change is made by another thread between when the `find_leaf` gets to the leaf and the validation is checked (a few instructions later). This means that unless there is significant contention at a single leaf (or its near neighborhood) most updates do not contend on locks, and do not fail their validation. TLF maintains the property of avoiding contention, as in the original data structure, by traversing outside of any locks and taking the same locks.

3 PRELIMINARIES

We assume a set of processes, and a set of shared memory locations that each process can access. Each process executes *local steps*, that consists of a sequence of reads and writes to the shared memory, as well as other step types that we will introduce. A scheduler interleaves the local steps of the processes. In particular, on each *global step* a scheduler chooses a local step from one of the processes. A *history* is the sequence of such global steps (i.e., an arbitrary interleaving of the local steps by each process). The history can thus be written as a sequence of steps, each of which can be indicated as $W_i(x)$ for a write to x by process i and $R_i(x)$ for a read from x by process i. When introducing more types of local steps, we will use a subscript to indicate from which process it comes from.

We assume every read of a variable x reads the value written by the prior write to that variable in the history. In a history, the local steps of two memory accesses *conflict* if one is a write to a variable and the other is a read or write of the same variable (e.g., $R_1(x)$ $W_2(x)$). When we say that steps are *equivalent*, we mean that their process, variable and value (read or written) are the same.

We consider protected regions of local steps of each process. Each *protected region* on process i starts with a step P_i (protect) and ends with a step U_i (unprotect). The steps in between correspond to sequences of steps that need to be executed atomically. Protected regions can be a transactions, but we will also use them to describe certain regions of an optimistic locking operation. We will later nest such protected regions inside a transaction as part of the TLF protocol. In general, a history might include both protected and unprotected regions. In the paper we say the *protected history* is the part of the history covered by protected regions (i.e., all unprotected steps are removed).

The *strict conflict graph* of a history is a directed graph where each vertex corresponds to a protected region, and there is an edge from the protected region A to protected region B if a step in A has a conflict with a later step (in the history) in B, or if U_A is before

P_B [3]. For a pair of protected regions A and B with an edge from A to B we say B *depends* on A and that A *precedes* B. Two regions are *ordered* if one is reachable from the other in the conflict graph, and *unordered* otherwise. A history is *strict conflict serializable* if there are no cycles in its strict conflict graph. For the rest of the paper, unless otherwise stated, we will use serializable to mean strict conflict serializable. Note that this definition does not consider the unprotected steps.

The set of histories that are strict conflict serializable is the same as the set for which the protected history can be reordered by swaps of adjacent steps in the history such that (1) no swap involves two conflicting operations, (2) no swaps involves two operations on the same process, (3) any U_i cannot swap with the next P_j, and (4) in the final order all steps in a protected region are adjacent [3].

4 OPTIMISTIC LOCKING

In this section we define the important properties of optimistic locking data structures. We start with a high-level description.

Broadly, operations on OL data structures consist of regions that are unprotected by locks and regions that are protected by locks. In practice, an unprotected region does some form of traversal, and each protected regions takes some locks, checks that the state is still valid, and if so makes an update or returns a value (for a query). The protected regions have two important properties. Firstly, the protected regions serialize, and, secondly, the writes performed by a protected region can depend only on the writes by regions that are ordered before it.

We consider data structures implemented with optimistic locking (OL). We say steps in an OL operation are *protected* if there is any lock held at the time of the step, and *unprotected* otherwise. A (un)protected region is a maximal sequence of adjacent steps that are (un)protected. An operation will consist of alternating unprotected and protected regions, and writes can only appear in a protected region. We assume the protected regions can fail (i.e., their validates do not pass). A failing protected region must not do any writes. Note that if a protected region fails, the operation will likely retry (although we don't require this). As described in the preliminaries, we will use P_i in the history to indicate the start of a protected region on process i and U_i to indicate the end of a protected on process i. We refer to a read in an unprotected region as an unprotected read.

Operations might have multiple protected regions even when they do not fail and retry. For example, in the insert for the optimistic B-tree we use, if on the search down the tree, a node is found to be too big or too small, the node is split or joined. This split or join is performed under locks, but then the locks are released and the operation continues to execute the insert in an unprotected region. This is similarly done in the AVL tree and Treap, where rotations are possibly performed in separate regions after the update.

Consider an operation Op and protected region A within it. We define the *important locations* of A as the locations read by A, and the *important reads* of A as all reads in A as well as all prior (within the operation) unprotected reads of important locations. Note that, in this definition, that not just the protected reads are important.

We say that an OL data structure is *stable* if it satisfies the following two properties:

Serializable. All histories that result from applying the operations of the data structure are serializable. Recall that our definition of serializable only includes the protected regions.

P-Dependent. Under all histories in which a protected region A of an operation has equal values for the important reads, A will perform the same steps.

We say an OL data structure is *strong stable*, if the data structure is linearizable and all operations linearize inside a protected region.

All OL data structures we have studied are stable, and are either strong stable, or easily converted to be so. Furthermore, and importantly, proving stability does not require proving correctness of the data structure, which can be quite complicated. It only requires assuming correctness of the original OL data structure and showing that it satisfies some properties that imply stability. This can be significantly simpler than proving correctness. Proving serializability is typically easy since the variables in the protected regions are appropriately protected by locks, e.g. using two-phase locking. The approach we will use to prove that OL operations are p-dependent is to show that there is only a single place in the data structure at which the operation can be applied given the protected regions that precede it. We refer to this as the *critical* location. For example, in a tree there could only be a single node under which we can apply an insert assuming its properties are maintained (e.g. that it is a BST). Furthermore, given this critical location, all we have to verify, e.g. by inspecting the code, is that the steps of the protected region only depend on that location, inputs to the operation, and important reads.

Here we prove that the leaftree implementation is stable, and in Section 8 we discuss the other data structures we consider.

Theorem 4.1. *Assuming that operations on a leaftree maintain the BST property on the internal nodes of the tree, the implementation is stable OL.*

Proof. The operations are serializable since they use two phase locking for their protected regions (starting with the first lock). We now argue why it is P-dependant. Consider an insert. The insert must fall between two keys that were inserted earlier, and the node we insert at (i.e., `parent` in the code) must be the one of these that was inserted later. This is because the internal nodes satisfy the BST property (parent must be either the key just before or just after) and we always insert at a leaf (i.e. the later will end up below the earlier). The `parent` is the critical location. Now, by inspection of the code, we note that all steps only depend on `k`, `parent`, `leaf` and values read in the protected region. The value of `k` is part of the operation, and the value of `parent` is fixed as just argued. The read of the `leaf` during the unprotected `find_leaf` is an important read since we read `leaf_pointer` on line 39. Hence the steps will only depend on the important reads.

A similar argument can be made for delete where the critical location is the `gnd_parent`. The variables `parent` and `leaf` read in `find_leaf` are important reads since the locations are also read, when validated, in the protected region. Therefore the code only depends on important reads (as well as `k` and `gnd_parent`). □

4.1 Some Additional Points

Here we note some subtle points about the definition of OL operations.

Firstly, as the reader might have noticed that our definition of OL operations does not make any reference to locks. It instead requires the protected regions to be serializable. In practice this is exactly what the locks in an implementation of OL operations do, typically using fine-grained two-phase locking, where some locks are acquired, the actions are taken and then the locks are released. For example, this is what the leaftree in Section 2 does. This also indicates that one way to run these OL operations is to ignore the specific locks, but only use the locks to determine when the code is in a protected region. Then in the protected regions all reads and writes can be serialized using an STM system. Using the locks to indicate protected regions is how one of our implementation techniques for TLF works, except that the transaction can span multiple operations.

Secondly, we note that p-dependent condition holds even if the protected region fails its validation—i.e. if it fails once when its important reads return the same value it will always fail with those same reads. This could happen, for example, if during the unprotected region we read a value from x and then during the following protected region P we read x again to validate that the value has not changed. If the first read was from a write to x in protected region A and the second from a write to x in protected region B, the validate could fail since the values are different. However in this case both reads of x are important since x is a critical location, and so if it fails in one history (due to the validation failure) it will fail in all histories with the same important reads.

Finally, we point out that for many data structures multiple concrete states (i.e., values stored in memory) can map to a single single abstract state. This could allow the steps in a protected region to depend on an unimportant read, but not be problematic as long as all outcomes have an equivalent abstract state. Our definition of stability disallows such an operation, but one could imagine a more lenient definition that instead of requiring that A perform the same steps, requires that under all histories, A does an equivalent transition on an abstract state. Even though most of the data structures we consider map multiple concrete states (e.g BST tree shapes) to the same concrete state (e.g. ordered set), we have not yet found a need for this flexibility. It is, however, an interesting topic for future research.

5 TLF SPECIFICATION AND CORRECTNESS

In this section, we define TLF without respect to a particular implementation of the transactional system. Instead we just put restrictions on the histories that the transactional system permits. This allows TLF to be integrated with a variety of STM systems including both optimistic and pessimistic, as well as both single version or multiversion. It, however, disallows any system that allows writing to a shared location and then aborting. For example, it disallows two-phase locking approaches that write before all locks are acquired, such as 2PLSF [34]. As discussed in Section 7, however, the 2PLSF algorithm can easily be modified to delay the writes so as to works with TLF.

In TLF, a transaction consists of a sequence of OL operations. The intent is to run these operations so that they appear to happen together atomically. The history of TLF transaction is the sequence of steps taken by the individual operations. It will therefore consist of the composition of all the unprotected and protected regions of each operation. Unlike standard transactions, the steps taken by the a TLF transaction may not be serializable due to unprotected reads.

In TLF, it is necessary that writes by an operation are visible to future operations within the same transaction, but not visible to operations in other transactions until the transaction commits. Being visible within the transaction is required so that each operation within the transaction can serialize after the others. Not being visible outside the transaction is required since (1) the unprotected reads can interleave arbitrarily in the history and (2) a transaction can abort after a write. Without the requirement, one of the unprotected reads could see a value of a transaction that then aborts, which leads to incorrect behavior.

To model writes that are only visible within a transaction, we add a *local write* memory operation. In a history this is indicated with an L (e.g. $L_1(x)$ is a local write to x on process 1). We will refer to the regular write, still indicated by a W, as a *shared write* for clarity. With local writes the semantics of a read $R_i(x)$ is as follows. Firstly, if there is a preceding local write to x within transaction i the read returns the value of this write, otherwise, the read returns the value of the preceding shared write in the history.

We say that a transactional system has *delayed writes* if every write to a variable x within a transaction i is split into a local write $L_i(x)$ at the position of the actual write in the code, and a shared write $W_i(x)$ that writes the same value, which is delayed to the end of the transaction, and only applied if the transaction succeeds. Furthermore the shared writes (if the transaction succeeds) are applied in the same order as the local writes in the code. We say a history of such a system is a *delayed-write* history. If the transaction aborts, then there are no shared writes. This is sometimes referred to as "write-back". In TLF we put the delayed writes into a protected region.

We define the *TLF conflict graph* of a history of TLF transactions as a directed graph where each vertex corresponds to a transaction, and there is an edge from transaction A to transaction B if a protected step A has a conflict with a later protected step in B, or the last step of A precedes the first step of B. We define *depends*, *precedes*, and *ordered* among transactions (vertices) as with a strict conflict graph. We say that a history of TLF transactions is *t-serializable* if there are no cycles in the TLF conflict graph.

Definition 1. *A TLF protocol is one that uses delayed writes and ensures all histories are t-serializable.*

We will prove that TLF transactions faithfully implement OL data structures, with the additional property that the operations within a transaction serialize together. Ideally we would prove this by showing that that every history generated by a TLF protocol on some set of OL operations is the same as some history generated by the the OL operations running in in isolation (without transactions). However, this is not true. Indeed without the local writes (i.e. replacing them with shared writes) the histories of TLF protocols do not correspond to a valid histories at all. We give an example of

```
T1:    L_A(x)      R_B(x) R_B(y)              ... W_A(x)
T2:         L_C(y)           R_D(y) R_D(x)       ... W_C(y)
```

Figure 8: Two transactions in the TLF protocol. The first one (T1) includes operations A, B and the second (T2) includes C and D. The operation B sees the new value of x and the old value of y while D sees the new value of y and the old value of x. This cannot be explained by a standard history, but can be with a history with local writes (as given).

such a history in Figure 8. We note, however, that for a TLF history to generate such a history, T_1 and T_2 have to be unordered and B's read of y and D's read of x must be unimportant, and hence inconsequential to the outcome of those operations. We take advantage of this property to prove correctness of TLF.

We say a *projected TLF history* is one in which we only consider the steps within a protected region, remove the shared writes, and replace each local write with a shared write. Recall that the protected history is the subset of steps in the history that are in a protected region.

Theorem 5.1. *For all histories H_t generated by the execution of a TLF protocol over any set of strong stable OL operations, there exists a history H_o of the same OL operations without TLF such that the projected TLF history of H_t is the same as the protected history of H_o. Furthermore all operations within each TLF transaction will linearize adjacently.*

Proof. t-serializability of TLF ensures we can place the protected regions of transactions in serialized order. Within each transaction the operations and hence protected regions will be in the order of execution (and all adjacent). Furthermore, all important reads for a protected region R in a transaction T will be of writes by protected regions that are ordered before R, either within the transaction or for transactions that are ordered before T. Hence the important reads will be consistent in the TLF and OL histories, and by the P-Dependent property they must both execute the same steps. With regards to linearizing adjacently, we note that due to the strong stable requirement every operation will linearize within a protected region and therefore maintain the linearized order. □

The theorem implies that if the OL operations are correct on their own, then when fused by the TLF protocol they are still correct.

6 STM REQUIREMENTS FOR USE WITH TLF

TLF can be implemented as part of many STM systems. We now describe what is required of a STM system to integrate it with TLF, and then outline how to do the integration. In the following, a *safe unprotected read* is a read within a transaction that properly receives the value of either the prior local write to its location within the transaction, if there is one, or the prior shared write in the history if not. Our requirements are:

(1) Transactions support safe unprotected reads.
(2) Transactions have delayed writes.
(3) All histories of the transactions are strict conflict serializable when the unprotected reads are removed and the whole transaction is marked as a single protected region.

Naïvely the first condition could be supported just by using transactional reads, although at no gain over a traditional STM. For many transactional systems, however, safe unprotected reads can be performed much more efficiently. For example, with single-version two-phase locking a safe unprotected read can be implemented with just a standard read as long as the read is protected from read-reclamation races (e.g., by using epoch-based memory management).

The second condition is true, or can easily be made true, by systems that implement delayed writes—i.e. that write to a write log during the transaction and only copy the write log to shared memory after they determine the transaction will succeed. Some STM systems that delay the writes might not write them to the shared locations in the same order as they were performed by the user code since if the STMs is serializable the write order does not make a difference. Out of order writes, however, is not an adequate implementation of delayed writes by our definition, and will lead to an incorrect implementation of TLF. However, it should not be hard to convert such an STM so writes are properly ordered.

For the third condition, strict conflict serializability is true for all STM systems we know of. However, some database transactional systems make weaker guarantees. For example in the related work we mentioned relaxed isolation levels. Also some systems are conflict serializable but are not strict. Such systems allow for stale read-only transactions—i.e., a read-only transaction might see a state that was valid at some prior time, but no longer valid when the transaction started. We do not guarantee that TLF works with such systems.

Given the conditions are satisfied there are two ways of implementing TLF within an STM system, one which ignores the specific locks taken by the OL operation, but just uses the locks to determine whether a step is in a protected region or not, and the other is to use the specific locks. In the first approach the system keep track of whether a region is protected by using a counter. The counter starts at zero, and whenever a lock is acquired the count is incremented, and whenever it is released the counter is decremented. If the counter is zero the region is unprotected, and otherwise it is protected. The locks are otherwise ignored. Now all reads in protected regions must be transactional reads, but all reads in unprotected regions can be unprotected reads.

The second approach requires more integration into the STM system and likely would require that the STM system itself uses locks. The idea in a pessimistic STM is that whenever a read or write lock is encountered the actual lock is taken and logged. The release of locks is then ignored, and instead the locks are held until the end of the transaction, when all are released. Now, since the reads and writes are already protected by the locks, the STM need not take any additional locks when encountering a read or write. All reads are therefore unprotected by the STM. The approach can also be applied an optimistic STM as described in the next section.

7 TWO EXPERIMENTAL TLF SYSTEMS

We have implemented two TLF systems that support the Fuse interface and generate TLF schedules. The same code can be run with both. The first is based on a state-of-the art two-phase locking STM, 2PLSF [34]. We make some modifications to support TLF as

described below. The second is a hand-rolled optimistic multiversion STM. It uses a standard optimistic protocol [15, 24], and uses versioned pointers [4] to avoid a level of indirection. We select these two STMs since they are on opposite ends of the spectrum—one using pessimistic concurrency with a single version, and the other optimistic concurrency with multiple versions. Also these both have state-of the art performance.

7.1 2PLSF

The 2PLSF STM uses traditional two-phase locking, but has an efficient approach to acquiring read locks and ensures starvation-freedom. The read locks only require setting a bit on a cache line that is never written by another processor. The starvation-freedom is implemented by a clever mechanism that only acquires timestamps when a transaction fails, and ensures that transactions are serviced in timestamp order. We maintain these features but make a handful of modest changes so as to work with TLF. We implement both variants outlined in the previous section—i.e., one that ignores the specific locks specified by the OL operations, and the other that uses them. Both require the following two modifications.

The first modification is to introduce an unprotected read that is just a standard read of an atomic location. However, this modification is not completely trivial since it introduces the possibility of read-delete races. For example, one transaction could delete a node while another is reading it, which is not safe. This is not possible in the original 2PLSF since all reads are protected by locks so the read cannot be concurrent with the delete. To avoid this problem we added epoch based reclamation, which protects against such read-delete races by delaying the delete.

Second, we modified 2PLSF to support delayed writes. A write-through approach is safe in 2PLSF, or generally in two-phase locking, since no readers can see the intermediate state of a transaction that eventually fails. In particular the writes that have been modified are locked. TLF requires delayed writes since the unprotected reads could be anywhere. We therefore modified 2PLSF to delay the writes. The writes were logged anyway since they need to be undone in the case of an abort. This also required modifying reads so they check the write log before going to the shared location.

The implementation that ignores the specific OL locks works as outlined in the previous section. It just uses the OL locks to increment (on acquire) and decrement (on release) a counter that is used to determine whether in a protected region or not. If a program read is in a protected region it uses the 2PLSF read (that takes a lock), and otherwise uses the unprotected read. The implementation that uses the specific OL locks also works as outline in the previous section. We use the 2PLSF implementation of read and write locks and hence preserve the fast read locks and starvation-free guarantees. We added a lock log. All reads become normal reads without locks, since the OL locks protect them. The writes also do not require locks since they are protected by the OL locks.

7.2 Optimistic Multiversioning

We implemented a multiversioned STM based on optimistic concurrency. We outline how the multiversion STM works, and then what is needed to include the TLF protocol. As with most multiversion STMs, we maintain a global timestamp. For each shared memory location we maintain a version list where each version contains the value of a write to the location and a timestamp for when it is written. The lists are sorted by timestamp, most recent first, allowing for efficient search of recent versions. We use the Verlib library [4] to implement the versioning, which allows us to avoid a level of indirection. For example, when the approach is used in a tree, every parent points directly to its child instead of to a head of a version list that points to the child, which can improve the performance by up to a factor of two.

In optimistic STMs, there is a speculative phase that runs the user code but all writes are local (perfect for TLF), and then a commit phase where the transaction validates the reads, and if successful commits the writes. In our version, as with most, a transaction starts by reading the global timestamp (the start time) and running the user code. During the speculative phase, when a write is encountered, its location and value are logged in a write log. When encountering a read, we check if it is in the write log, and if not, find the version in the version list at the start time. In our implementation, if the location is write locked, the read waits until it is released so as to ensure it is not being modified while being read. The read is also inserted into a validate log for later validation. All reads during the speculative phase will see a snapshot of the state at the start time.

In the commit phase, when the user code is complete, locks are taken on all the write locations. The global stamp is then incremented, and the value it is incremented to is kept as the commit time. Then all entries in the read log are validated by checking that the most recent timestamp of each is no larger than the start time (i.e. the location has been written since the start of the transaction). If all validates succeed, the transaction has succeeded and each write is added to the head of its version list. If not, the transaction aborts. In either case the locks are then released.

The modifications for TLF are rather simple. For the version that ignores the specific OL locks, the locks are just used for incrementing and decrementing the "in protected" counter. Any reads that are in a protected region are added to the validate log, but ones that are in unprotected regions are not added. Otherwise the implementation is unchanged. For the version that uses the OL locks, we do not acquire the OL lock when encountered, but keep a lock-log of just the OL write locks. During the commit phase, instead of taking locks on all the written locations, we take a lock on the OL locks, and these locks are released at the end of the commit phase. As before, reads are only added to the validate log if they in a protected region. Otherwise the implementation is the same as the non TLF version.

8 DATA STRUCTURES

In this section, we briefly describe the data structures we use in our experiments and point out the critical locations that imply they are strong stable OL data structures. Actual proofs that they are strong stable would require a complete description of the algorithm and inspection of the code. These structures are all taken from the Verlib library [4] and are implemented in C++. All structures support find, insert and remove. Furthermore, in almost all cases, the traversal (unprotected) phase is identical for the three operation and only

the protected phases differ. Most of them maintain a `deleted` flag that is used during validation.

We have already described the leaftree. The ordered linked list structure is the simplest. The traversal phase follows the list to the right spot. It then takes locks, checks it is in the right spot and does its operation. If it is a remove, then it needs to take two locks. The critical location is the node just before the one to be inserted, removed or found. The treap and AVL trees are both balanced trees that involve rotations. When making updates they both work with two passes over the tree, one that does the update and the other that rebalances if necessary. Each of these involves an unprotected traversal phase and then possibly a protected update phase. In both cases the critical location is the parent (or possibly grandparent for removes) of where the change will take place. For the rebalance it is important that the nodes being rebalanced are copied and then reinstalled with a single write. This is so unprotected readers will not see an inconsistent partial rotation.

The skiplist is a relatively standard skiplist. When updating at level i of a stack (level zero is the bottom level, which includes all the keys), the search to that level is unprotected, and then all levels below it (typically only one or two) are locked and modified. Insertions are performed bottom up and deletions top down. This is important for correctness since the unprotected reads can see the state while updates are happening. Inserting top down or deleting bottom up leads to an inconsistent view. This is a case where having delayed writes in TLF happen in the correct order is important. The critical location is the topmost location in a stack that is locked.

The Btree is a top-down variant that ensures nodes are not over or underfull on the way down. In particular it has a maximum and minimum size and if while traversing the tree to a leaf in unprotected mode encounters a node with maximum or minimum size, it takes locks and splits or merges the node. It then releases the locks and starts again at the root. When it reaches a leaf it finds, inserts or removes the key from the leaf. The slot in the parent that points to either the leaf or the underfull/overfull node is the critical location. The ART tree (adaptive radix tree) [25] is a radix-based tree with different kinds of nodes depending on how full they are. It's implementation is similar to the Btree (i.e. fixing on the way down). The hash table uses "separate chaining", but each chain is stored as a block. The critical location is the bucket.

9 RELATED WORK

There has been significant work on incorporating higher-level operations directly into transactional systems with the goal of using existing concurrent data structures or reducing unnecessary conflicts.

Transactional boosting [19, 36, 40] allows using existing implementation of a concurrent data structure within a transaction for data structures that have undo operations and support "semantic" locks. For example, for a dictionary, a delete could undo an insert (or insert a delete), and the semantic lock could be on the value of the key. When an operation is applied in a transaction, the approach first takes a "semantic" lock on the operation (e.g. the key). Once the lock is taken the approach applies the concurrent operation immediately to the data structure (e.g. an insert into a b-tree). The semantic lock prevents any other conflicting operation (e.g. inserts, searches or deletes on the same key) from seeing the update or being applied until the semantic lock is released. If a transaction aborts, all operations that were applied during the transaction are undone (e.g. if an insert was applied, a delete will remove the element), and the locks are released. If the transaction succeeds the locks are released and the updates become visible.

Transactional boosting is limited in its use by the need for an undo operation and the need for some semantic handle on which to apply the semantic lock. It also has other disadvantages. Firstly, it requires two levels of synchronization–one for the semantic lock and one in the concurrent structure itself. Secondly, it requires taking locks during the speculative phase, and therefore is mostly limited to pessimistic two-phase locking as opposed to optimistic concurrency. Thirdly, it is not clear how it can be extended to operations such as range queries, or other multi-point queries. Indeed even an operation that returns the largest key less than the query key seems difficult (on what key is the semantic lock taken?).

A variant on transactional boosting is transactional predication [6], which is designed to work with a broader set of transactional systems, including optimistic ones. The approach, however, is designed specifically for dictionaries. The idea is to map the keys of a concurrent dictionary to memory locations, each of which is either empty or full with a value. When in a transaction operations first try to find their key in the concurrent dictionary, and if not present they add the key along with a newly allocated location. Once the key is inserted into the dictionary, an Insert(k, v) operation can be implemented by writing `Full`(v) to the location associated with k using a transactional write. Similarly, delete(k) can be implemented with a transactional write of `Empty` to k's location, and find(k) by a transactional read on k's location. Therefore the idea can be mixed with just about any transactional memory system. Although more general than transactional boosting in terms of what transactional systems it can be used in, it is even more limited in the data types it can support. Also, as with boosting, implementing other operations atomically, such as range searching or previous, does not seem to fit the paradigm.

Another approach for supporting higher-level operations is to supply hooks into the transactional system so that a data structure designer can specify what should be implemented during each of the phases of the transactional system (e.g. speculative phase, locking-phase and validate phase). Optimistic Transactional Boosting (OTB) [16, 17] uses this approach. Motivated by optimistic locking they divide the code into parts that are run during the read phase, lock phase, and validate phase [check]— see e.g. [17, figures 1, 2 and 3] which gives such code for an ordered linked list based on lazy lists [18]. The code ends up looking quite different from the original lock-based code since it includes a considerable amount of code that is needed for the transactions. Herman et al. [21] and Spiegelman et al. [38] use similar approaches, but with more phases. For example, in Herman et al., the user can supply code to run during five phases including a cleanup phase. The main disadvantage of all these approaches is that they require special purpose code for each data structure—they do not use black-box implementations of existing concurrent data structures. The approach of Herman et al. is used in the STO database system [22], which we compare to in our experiments.

Another approach is to use open nested transactions [30]. This allows one to nest transactions and have inner transactions act somewhat independently of the enclosing transaction (e.g. aborting without aborting the enclosing transaction, or having permanent effects even if the enclosing transaction ends up aborting). This can be used to implement many of the ideas mentioned above, but is an extremely complicated and error prone interface for a user.

Recent work has studied taking CAS based implementations of lock-free data structures and allowing them to be composed so they act atomically [7, 42]. LFTT (lock free transactional transform) [42] composes a certain class of lock-free CAS-based structures into lock-free transactions. It requires identifying a critical node for each operation. This is similar to the restriction of requiring logical-locks for transactional boosting. It also requires significant refactoring and modification from an original CAS-based implementation, including separating the logical from the physical status of the data structure.

NBTC (nonblocking transactional composition) [7] is perhaps the most similar to our work, modulo the fact that it is based on lock-free CAS-based implementations of data structures rather than of lock-based structures. Its goals, like ours, are to be applicable to a broad set of structures while making minimal changes to existing implementations. It, however, still requires annotating all CASes with whether they are linearization points and/or publication points. Furthermore the user needs to add locations to a read set by hand, and specify code for a cleanup phase. None of these are required in TLF. In terms of restrictions, NBTC requires that operations have an "immediately identifiable" linearization point. It seems this covers a reasonably broad set of structures. NBTC does not use multiversioning so its read-only transactions can abort. It also is only obstruction free instead of lock free. We compare performance to NBTC in our experiments.

The database community has considered various forms of relaxed reads that violate serializability [2]. The level to which the reads are relaxed are referred to as isolation levels. In particular the ANSI standard for SQL considers three isolation levels that are relevant to STMs (read uncommitted, read committed and serializable) and a fourth that is only relevant to databases (phantom reads). A uncommitted read can read values of transactions that will later abort, and a committed read will only read values between transactions. Neither of these correspond to the unprotected reads we consider since our unprotected reads can read values in the middle of transactions, but not if they abort. Also in these isolation levels all reads in a transaction follow the same protocol, while here some are protected (serializable) and some unprotected. Non-serializable reads have also been considered in the STM community [9, 20]. In this work read accesses can be protected for a limited region, and correspond to read-committed accesses in the ANSI standard. We know of no work that has systematically studied how to use relaxed reads in the design of data structures.

Kung and Lehman introduced optimistic locking in 1980 [23] and applied it to concurrent balanced binary search trees supporting insertions, deletions, rotations and finds. It has since been widely used in the design of lock-based concurrent data structures [1, 5, 14, 18, 26, 32]. At least two libraries, OPTIK [14] and flock [1], are mostly based on data structures based on optimistic locking.

10 EXPERIMENTS

We apply TLF to a variety of data structures and workloads, demonstrating that this approach can significantly improve the performance of transactional operations for both STM systems. Further, the 2PLSF paper compares its system to other state-of-the-art transactional systems like OneFile [35], TinySTM, and TL2 [12]. It shows that 2PLSF outperforms these systems in benchmarks with a high percentage of writes, and is at worst marginally slower in read-dominated benchmarks. Additionally, we compare to STO [22], another transactional system that also allows for avoiding aborts due to non-semantic conflicts.

Setup. All experiments, except those to compare to STO, ran on a 96-core Amazon Web Service c7i-metal instance with 2x Intel(R) Xeon(R) Platinum 8488C (48 cores and 3.2GHz), and 384GB memory. Each core is 2-way hyperthreaded, giving 192 hyperthreads. The STO experiments were run on a 1 node c7i instance with the same processor specifications. We used `numactl -i all`, evenly spreading the memory pages across the sockets in a round-robin fashion. The machine runs Ubuntu 22.04.1 LTS. The code was compiled with g++11 with `-O3`. We report on the 8 data structures described in previous section.

Workloads. Our experiments are based on the YCSB benchmark suite, which is commonly used to benchmark key-value stores. In these experiments, we vary the following parameters in different experiments: (a) data structure size (denoted by n), (b) update percentage, (c) size of transaction (i.e., number of operations in each transaction), (d) number of threads, (e) zipfian parameter. In all experiments we use a universe U of $2n$ distinct, uniform random 64-bit keys. We associate with each a 64-bit value (a total of 16 bytes for the key and value) We initialize the structures by inserting n of these keys and then start the experiment.

In the timed portion of the code, each thread performs a mix of operations, consisting of inserts and deletes (done in equal numbers), and finds. These operations are batched into groups of the transaction size. Each group will contain a mix of the different operation types. Keys for all operations (including initialization) are drawn randomly from U, which ensures that the size of the data structure remains approximately n throughout the experiment. Keys are drawn from Zipfian distribution with parameter ranging from 0 (uniform distribution) to .99 (highly skewed). The Zipfian distribution is used in the YCSB benchmark [10] to model real world access patterns.

When unspecified, we fix every parameter at their default values. The default n for the linked list benchmarks is 300, while for all other data structures it is 10^5. The default zipfian is 0.9, the default transaction size is 4, the default update rate is 5%, and the default number of threads is 191.

TLF and STM systems tested. As previously noted, user-specified locks can either be ignored or taken when applying TLF to an STM system. We found that 2PLSF generally performed best when augmented to ignore user-specified locks and instead take locks for each memory location written. This is because 2PLSF is more sensitive to contention, which finer-grained locking reduces.

The multiversioned STM, in contrast, generally performed best when augmented to take the locks specified in the OL code. This is because multiversioning allows this system to be less sensitive to

contention, and the overhead from taking more finer-grained locks outweighs the benefits.

We denote these augmented implementations as 2PLSF-TLF-STM and MV-TLF, respectively, and report their performance in the following results, omitting those for the alternative implementations. We also benchmark 2PLSF and the multiversioned STM without TLF, which we denote as 2PLSF-STM and MV-STM. For measurements with these two STM implementations, we took the eight OL data structures, removed all locks, all validates, and all retry loops. This also allowed us to remove the `deleted` flags since they are just used for the validation.

10.1 Geometric Mean Performance

Our first set of experiments try to understand the effect of TLF across a wide variety of workloads. Specifically for every data structure and STM implementation (2PLSF, 2PLSF-STM-TLF, MV-STM, and MV-TLF), we record the throughput while varying the parameters among the following values:

- Update Percentage $\in \{0\%, 5\%, 50\%\}$
- $n \in \{10^2, 10^3\}$ for linked list, $n \in \{10^4, 10^6, 10^8\}$ for all other structures
- Transaction Size $\in \{1, 16\}$
- Zipfian $\in \{0, 0.99\}$

We then take the geometric mean of throughput across these workloads. Figure 1 in the introduction displays the results. In the bar chart, the throughput of each group is normalized to the fastest among them. For these runs, we fix the number of threads at 191.

As the bar chart shows, TLF improves throughput for every data structure and for both STM systems (2PLSF and MV), though this improvement is not uniform across the data structures. In particular, the disparity is much greater for data structures with long unprotected traversals, especially the linked list. This is to be expected, both because unprotected reads are cheaper, but more importantly because using unprotected reads greatly reduces the number of aborts.[2] For example, in the list when using two-phase-locking every read will take a read lock blocking any updates from occurring along the path. Similarly, with optimistic locking all the reads will be validated and many will abort since there will likely be an update at some of the locations along the path. The net result is that TLF improves performance on the binary trees by more than a factor of 10 when averaged (geometric mean) across the workloads and almost by 100 on specific workloads.

The performance improvement due to TLF is least on the data structures with short traversals and wide fan-out, and in particular the ART tree, B-tree and hash table. Here the unprotected traversals are short and the operations are scattered widely creating little contention between updates and unprotected reads. The improvements on these range from 5% to 20%. The binary trees (AVL tree, Leaf Tree, and Treap), and the skip list, fall in between since these have modest length unprotected traversals ($O(\log n)$) and only binary forking. The overall performance improvement for these ranges from 30% (leaftree with MV) to more than a factor of two (treap with 2PLSF).

[2]If the reviewers feel it would be helpful to present number of aborts, we have the numbers, but they pretty much align with performance.

10.2 Varying Parameters

Our next set of experiments measures the effect on our TLF library of varying each of the five YCSB workload parameters. The results are shown in Figures 9 and 10. The goal is to better understand how the various workloads affect the relative performance. In all the graphs throughput, measured in millions of operations per second, is reported on the y-axis. We present six of the data structures.

Varying Size. Row 1 compares throughput for varying size. We see that, depending on the data structure, sometimes TLF does best on small sizes (e.g., the skip list) and sometimes on medium sizes (e.g., the treap), and sometimes larger sizes (e.g., the AVL tree with MV). We see in Figure 9 (row 1, column 3) that throughput of MV-TLF suffers at low sizes for the B-tree. This is because of the B-tree's relatively wide fanout and in turn shallow depth. The user-specified locks in the OL code are too coarse-grained, resulting in the root being highly contended at low sizes. In this case, ignoring the OL locks and relying on the STM writes would result in lower contention.

Varying thread count. Row 2 compares the throughput of each STM implementation under various thread counts. We see that all algorithms implemented with MV-STM and MV-TLF scale well up to the number of hyper-threads on our machine (192). However, we see that 2PLSF and 2PLSF-TLF-STM do not scale well past 96 threads, with the throughput increasing negligibly from 96 to 191 threads. This is because their writer locks are not scalable. In particular, for p threads, taking a write lock on a location requires scanning p read indicators to determine whether a read lock has been taken on that location. Comparing STM to TLF we see that TLF generally does better at higher thread counts. This is likely due to the fact that TLF reduces contention among thread by avoiding synchronizing the unprotected reads. This is particularly noticeable with the linked lists.

Varying skew. Row 3 compares throughput for varying the skew of the input distribution. Higher Zipfian distributions means that the workload is more skewed towards certain keys which increases contention. We note that the relative performance of TLF and STM does not change very much across the spectrum. Although this might seem to contradict the statement under scalability about TLF doing better under contention, we note that the type of contention is different. For high skew, there is contention at the critical locations (e.g. the leaves) which both TLF and STM protect. For high thread count, however, the contention is also along the paths, which has more effect on STM. Both 2PLSF systems perform poorly at high skew, as again 2PLSF suffers at high contention.

Varying transaction size. Row 4 compares the throughput for various transaction sizes. The throughput per operation for all systems generally decreases as the number of operations increases. This is because the zipfian parameter is set to .9, which creates non-negligible contention. Since all operations within a transaction need to serialize, if any of them fails due to contention, they all fail. Under a uniform distribution the lines are relatively flat. Additionally, we see that 2PLSF-TLF performs exceptionally well at 1 operation per transaction. This is because no read locks are required when there is a single operation. In contrast, the original 2PLSF (2PLSF-STM)

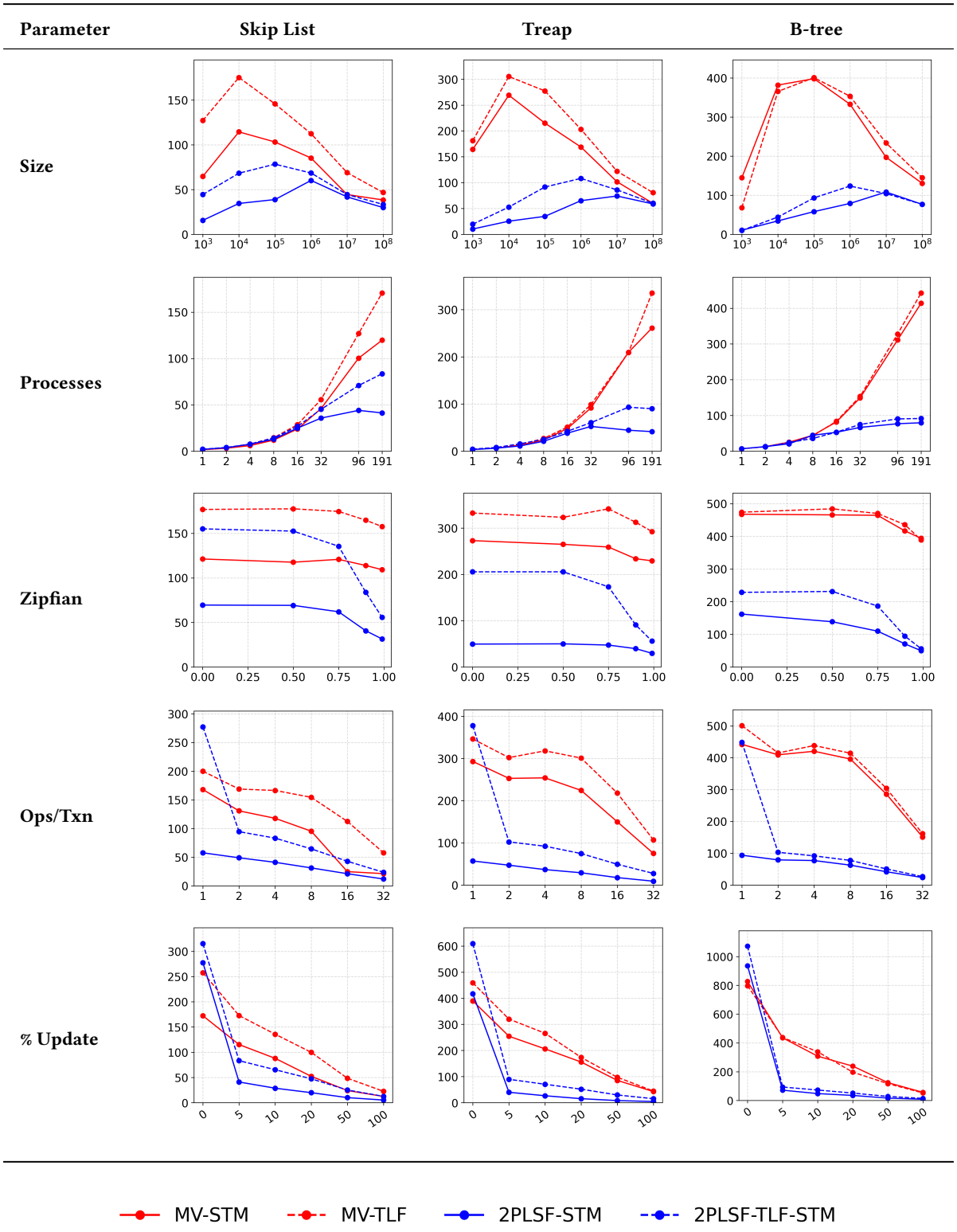


Figure 9: Throughput results for parameters across Skip List, Treap, and B-tree. For every plot, throughput in (MOps/sec) is measured on the y-axis, while the respective parameter is varied on the x-axis. For the variables that are not varying, the default for size is 10^5, the default processors is 191, the default zipfian is 0.9, the default ops/txn is 4 and the default update rate is 5%.

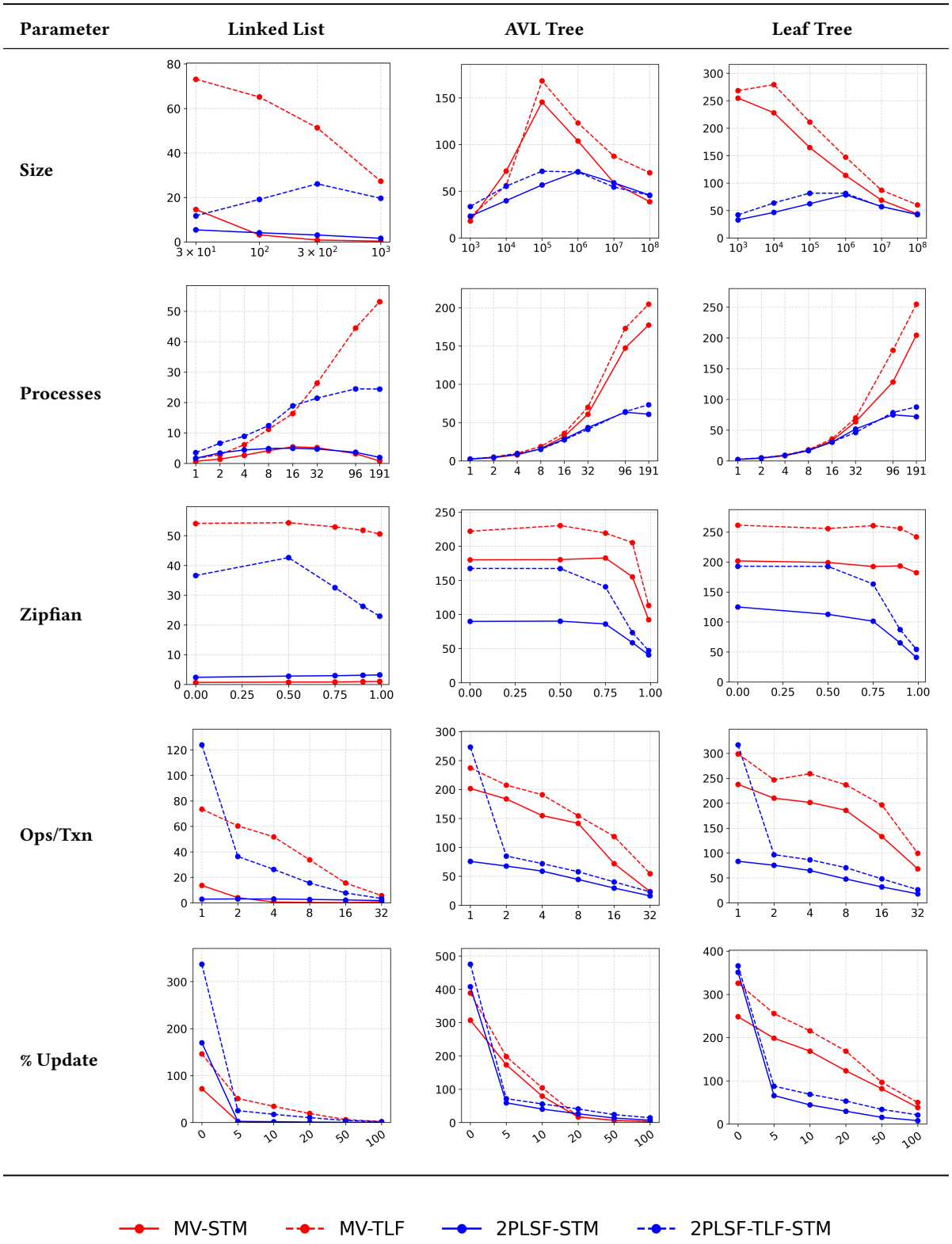


Figure 10: Throughput results for parameters across Linked List, AVL Tree, and Leaf Tree. For every plot, throughput in (MOps/sec) is measured on the y-axis, while the respective parameter is varied on the x-axis. For the variables that are not varying, the default for size is 10^5 (except for linked lists, where it is 300), the default processors is 191, the default zipfian is 0.9, the default ops/txn is 4 and the default update rate is 5%.

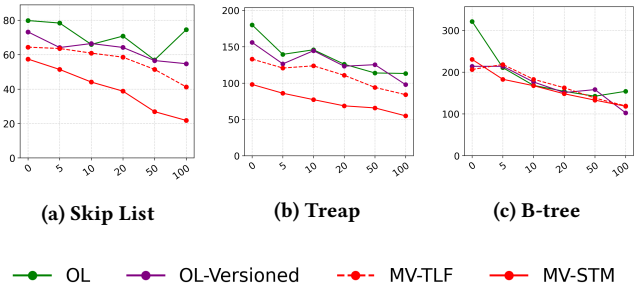
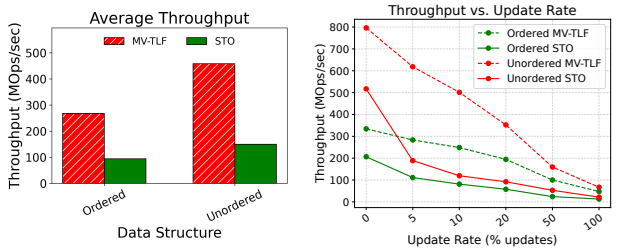


Figure 11: Comparison to OL code for Skip List, Treap, and B-tree. Throughput in MOps/sec is measured on the y-axis, while update percentage is varied on the x-axis.



(a) Geometric Means (b) Varying Update Rate

Figure 12: Throughput comparison of MV-TLF vs. STO on unordered and ordered map implementations.

must take a read lock for every read, increasing the time these locks are held. This is required since memory reads are not protected.

Varying Update Rate. Row 5 measures the throughput for various update rates. The left-most data points show a read-only workload, and the right-most an update-only workload. These experiments indicate that TLF does better than STM all the way from read-only to update-only. We also see that 2PLSF performs well at low update rates. This is because 2PLSF performs single-versioned reads, while MV-STM performs multi-versioned reads. Thus, the overhead from traversing version lists dominates. However, the throughput of 2PLSF and 2PLSF-TLF-STM drops off quickly as the update frequency increases. This is because, again, 2PLSF suffers at high contention.

Comparison to Original Optimistic Locking Code. In Figure 11 we compare to the original OL code, and the original OL code augmented with multiversioning (OL-Versioned), across varying update percentages. The goal here is to see how well the transactional code performs compared to the raw concurrent data structure. The TLF data structures were run with transactions of 16 operations (or 2 operations for the linked list). The non-transactional data structures were run with non-atomic "groups" of the same number of operations. We fixed n at 10^7, the zipfian at 0, and all other parameters at their default values. We set the zipfian at 0 because we wish to measure the overhead due to multiversioning and transactions. Increasing the zipfian would increase contention, and the likelihood of aborts for the transactional systems. The throughput of the locking code would be less affected by higher contention, precluding a fair comparison. Overall, across all data structures, we see that TLF introduces about 20% overhead compared to the locking code. About half of this overhead is due to multiversioning, and the other half is due to the transactional system.

10.3 Comparison to STO

Finally, we compare to the performance of STO [22] on these YCSB workloads. STO is a recent database system with state-of-the-art performance. It supports two index types, an unordered index based on hash tables and an ordered map based on Masstrees [28]. As discussed in the related work, it avoids some of the overheads of STM by integrating the structures directly into the transactional system. We compare the throughput of their unordered index and ordered index with our hash table and ART tree (both implemented with MV-TLF). Although the implementations of unordered and ordered maps support the same functionality using similar data structures (hash tables and radix trees), this is not quite an apples-to-apples comparison since the systems are different. It does, however, show that generalized STM is competitive with specialized database systems. We use their single version optimistic (OCC) system, which performs the best among their concurrency control protocols. Their system only supports up to 128 threads so we run the experiments on a single node of our system with 96 threads. For one experiment, we evaluate the throughputs of these data structures across the same parameter mix as in Section 10.1. The geometric means of these throughputs are reported in Figure 12a. Across these parameters, the datastructures implemented with MV-STM notably outperform their counterparts implemented using STO.

Additionally, we evaluate the performance of these data structures under varying contention by measuring their throughputs over different update rates. We report these results in Figure 12b, fixing the transaction size at 16, zipfian at 0.9, and size at 10^8.

11 CONCLUSION

The TLF methodology presents a general framework for modifying STM systems to avoid conflicts due to non-semantic aborts, provided they satisfy a set of reasonable requirements. This is particularly useful for transactionally composing operations on OL data structures with traversal phases consisting of sequences of unprotected reads. In order to justify the correctness of our methodology, we identify the stability property of optimistic locking data structures, and prove that TLF is sound for data structures respecting this property. We then apply TLF to two STM systems and evaluate them on realistic YCSB-like benchmarks, demonstrating that TLF significantly improves performance for OL data structures with long traversal phases and does not add significant overhead to the original OL code. Our proofs and evaluation together show that the TLF methodology is both correct and practically viable.

ACKNOWLEDGMENTS

We thank the anonymous referees for their comments. This work was supported by the National Science Foundation grants CCF-1901381, CCF-1910030, and CCF-1919223.

REFERENCES

[1] Naama Ben-David, Guy Blelloch, and Yuanhao Wei. 2022. Lock-Free Locks Revisited. In *ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*.

[2] Hal Berenson, Phil Bernstein, Jim Gray, Jim Melton, Elizabeth O'Neil, and Patrick O'Neil. 1995. A critique of ANSI SQL isolation levels. *ACM SIGMOD International Conference on Management of Data (SIGMOD)* 24, 2 (1995). https://doi.org/10.1145/568271.223785

[3] Philip A. Bernstein, Vassos Hadzilacos, and Nathan Goodman. 1987. *Concurrency Control and Recovery in Database Systems.* Addison-Wesley.

[4] Guy E. Blelloch and Yuanhao Wei. 2024. VERLIB: Concurrent Versioned Pointers. In *ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*.

[5] Nathan G. Bronson, Jared Casper, Hassan Chafi, and Kunle Olukotun. 2010. A Practical Concurrent Binary Search Tree. In *ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*.

[6] Nathan G. Bronson, Jared Casper, Hassan Chafi, and Kunle Olukotun. 2010. Transactional Predication: High-Performance Concurrent Sets and Maps for STM. In *ACM Symposium on Principles of Distributed Computing (PODC)*. https://doi.org/10.1145/1835698.1835703

[7] Wentao Cai, Haosen Wen, and Michael L. Scott. 2023. Transactional Composition of Nonblocking Data Structures. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. https://doi.org/10.1145/3558481.3591079

[8] Calin Cascaval, Colin Blundell, Maged Michael, Harold W. Cain, Peng Wu, Stefanie Chiras, and Siddhartha Chatterjee. 2008. Software Transactional Memory: Why Is It Only a Research Toy? The promise of STM may likely be undermined by its overheads and workload applicabilities. *Queue* 6, 5 (2008).

[9] Christopher Cole and Maurice Herlihy. 2005. Snapshots and software transactional memory. *Sci. Comput. Program.* 58, 3 (2005). https://doi.org/10.1016/J.SCICO.2005.03.006

[10] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking Cloud Serving Systems with YCSB. In *Proc. 1st ACM Symposium on Cloud Computing*. https://doi.org/10.1145/1807128.1807152

[11] Tudor David, Rachid Guerraoui, and Vasileios Trigonakis. 2015. Asynchronized concurrency: The secret to scaling concurrent search data structures. *ACM SIGARCH Computer Architecture News* 43, 1 (2015).

[12] Dave Dice, Ori Shalev, and Nir Shavit. 2006. Transactional Locking II. In *International Symposium on Distributed Computing (DISC)*.

[13] Guy Golan-Gueta, G. Ramalingam, Mooly Sagiv, and Eran Yahav. 2017. Automatic Scalable Atomicity via Semantic Locking. *ACM Transactions on Parallel Computing (TOPC)* 3, 4, Article 21 (mar 2017). https://doi.org/10.1145/3040223

[14] Rachid Guerraoui and Vasileios Trigonakis. 2016. Optimistic concurrency with OPTIK. In *ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*, Rafael Asenjo and Tim Harris (Eds.). https://doi.org/10.1145/2851141.2851146

[15] Tim Harris, James Larus, and Ravi Rajwar. 2010. *Transactional Memory, 2nd Edition.* Morgan and Claypool Publishers.

[16] Ahmed Hassan, Roberto Palmieri, Sebastiano Peluso, and Binoy Ravindran. 2017. Optimistic Transactional Boosting. *IEEE Transactions on Parallel and Distributed Systems* 28, 12 (2017). https://doi.org/10.1109/TPDS.2017.2725837

[17] Ahmed Hassan, Roberto Palmieri, and Binoy Ravindran. 2014. On Developing Optimistic Transactional Lazy Set. In *Conf. on Principles of Distributed Systems (OPODIS)*.

[18] Steve Heller, Maurice Herlihy, Victor Luchangco, Mark Moir, William N. Scherer, and Nir Shavit. 2006. A Lazy Concurrent List-Based Set Algorithm. In *Conf. on Principles of Distributed Systems (OPODIS)*.

[19] Maurice Herlihy and Eric Koskinen. 2008. Transactional Boosting: A Methodology for Highly-Concurrent Transactional Objects. In *ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*.

[20] Maurice Herlihy, Victor Luchangco, Mark Moir, and William N. Scherer. 2003. Software Transactional Memory for Dynamic-Sized Data Structures. In *ACM Symposium on Principles of Distributed Computing (PODC)*.

[21] Nathaniel Herman, Jeevana Priya Inala, Yihe Huang, Lillian Tsai, Eddie Kohler, Barbara Liskov, and Liuba Shrira. 2016. Type-Aware Transactions for Faster Concurrent Code. In *ACM European Conference on Computer Systems (EuroSys)*. https://doi.org/10.1145/2901318.2901348

[22] Yihe Huang, William Qian, Eddie Kohler, Barbara Liskov, and Liuba Shrira. 2022. Opportunities for optimism in contended main-memory multicore transactions. *VLDB J.* 31, 6 (2022). https://doi.org/10.1007/S00778-021-00719-9

[23] H. T. Kung and Philip L. Lehman. 1980. Concurrent Manipulation of Binary Search Trees. *ACM Trans. Database Syst.* 5, 3 (1980).

[24] H. T. Kung and John T. Robinson. 1981. On Optimistic Methods for Concurrency Control. *ACM Trans. Database Syst.* 6, 2 (1981).

[25] Viktor Leis, Alfons Kemper, and Thomas Neumann. 2013. The Adaptive Radix Tree: ARTful Indexing for Main-Memory Databases. In *IEEE International Conference on Data Engineering (ICDE)*.

[26] Viktor Leis, Florian Scheibner, Alfons Kemper, and Thomas Neumann. 2016. The ART of Practical Synchronization. In *Proc. International Workshop on Data Management on New Hardware*.

[27] Li Lu and Michael L Scott. 2013. Generic multiversion STM. In *Proc. International Symposium on Distributed Computing*. Springer.

[28] Yandong Mao, Eddie Kohler, and Robert Tappan Morris. 2012. Cache Craftiness for Fast Multicore Key-Value Storage. In *ACM European Conference on Computer Systems (EuroSys)*.

[29] Thomas Neumann, Tobias Mühlbauer, and Alfons Kemper. 2015. Fast Serializable Multi-version Concurrency Control for Main-Memory Database Systems. In *ACM SIGMOD International Conference on Management of Data (SIGMOD)*.

[30] Yang Ni, Vijay S. Menon, Ali-Reza Adl-Tabatabai, Antony L. Hosking, Richard L. Hudson, J. Eliot B. Moss, Bratin Saha, and Tatiana Shpeisman. 2007. Open Nesting in Software Transactional Memory. In *ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*. https://doi.org/10.1145/1229428.1229442

[31] Dmitri Perelman, Anton Byshevsky, Oleg Litmanovich, and Idit Keidar. 2011. SMV: Selective Multi-Versioning STM. In *Proc. International Symposium on Distributed Computing*.

[32] William Pugh. 1989. *Concurrent Maintenance of Skip Lists.* Technical Report TR-CS-2222. Dept. of Computer Science, University of Maryland, College Park.

[33] Pedro Ramalhete and Andreia Correia. 2024. Scaling Up Transactions with Slower Clocks. In *ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*.

[34] Pedro Ramalhete, Andreia Correia, and Pascal Felber. 2023. 2PLSF: Two-Phase Locking with Starvation-Freedom. In *ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*.

[35] Pedro Ramalhete, Andreia Correia, Pascal Felber, and Nachshon Cohen. 2019. OneFile: A wait-free persistent transactional memory. In *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*.

[36] Peter M. Schwarz and Alfred Z. Spector. 1984. Synchronizing Shared Abstract Types. *ACM Transactions on Computer Systems (TOCS)* 2, 3 (aug 1984).

[37] Michael F. Spear, Luke Dalessandro, Virendra J. Marathe, and Michael L. Scott. 2009. A comprehensive strategy for contention management in software transactional memory. In *ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*, Vol. 44. Association for Computing Machinery. https://doi.org/10.1145/1594835.1504199

[38] Alexander Spiegelman, Guy Golan-Gueta, and Idit Keidar. 2016. Transactional Data Structure Libraries. In *ACM Conference on Programming Language Design and Implementation (PLDI)*. https://doi.org/10.1145/2908080.2908112

[39] Yuanhao Wei, Guy E. Blelloch, Panagiota Fatourou, and Eric Ruppert. 2023. Practically and Theoretically Efficient Garbage Collection for Multiversioning. In *ACM Symposium on Principles and Practice of Parallel Programming (PPOPP)*.

[40] W.E. Weihl. 1988. Commutativity-based concurrency control for abstract data types. *IEEE Trans. on Comput.* 37, 12 (1988). https://doi.org/10.1109/12.9728

[41] Yingjun Wu, Joy Arulraj, Jiexi Lin, Ran Xian, and Andrew Pavlo. 2017. An Empirical Evaluation of In-memory Multi-version Concurrency Control. *Proceedings of the VLDB Endowment (PVLDB)* 10, 7 (March 2017).

[42] Deli Zhang, Pierre Laborde, Lance Lebanoff, and Damian Dechev. 2018. Lock-Free Transactional Transformation for Linked Data Structures. *ACM Transactions on Parallel Computing (TOPC)* 5, 1, Article 6 (June 2018). https://doi.org/10.1145/3209690